SHEET 02

C Strings & I/O

Station names on the CB radio, fault codes on the dash, the trip log on disk: text and I/O are where C quietly overflows or loses data.

Debug build:
`-std=c17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2 · -fanalyzer · -fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▶ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 12 entries

C-STR-01 strcpy into a fixed buffer

SR-404 · Radio / CB station name (Ep 7)

✕ BUGGY
```c
char station[8];
void set_station(const char *name) {
    strcpy(station, name);   // "KRAD 101.5" needs 11 bytes
}
```

▶ ASan: global-buffer-overflow

What goes wrong: strcpy copies through the terminating null and takes no destination capacity. Exceeding the destination violates its bounds; C does not promise neighboring-global corruption or a particular radio effect.

> FIX
```c
char station[8];
int set_station(const char *name) {
    if (name == NULL) return -1;
    size_t n = strlen(name);
    if (n >= sizeof station) return -1;
    memmove(station, name, n + 1); // overlap allowed, includes null
    return 0;
}
```

Sources: C11 draft 7.24.2.2-7.24.2.3;7.21.6.5; glibc manual Levels1-3, unknown sizes and SIGABRT failure; Host example capture C-STR-01 exact supplied-example diagnostics/results; Defined host checks C-STR-01 defined inputs/error policy; host_abi observations

Avoid it: For this reject-without-changing policy, validate length before writing. The input must be a valid terminated string; memmove permits overlap. snprintf can modify/truncate before returning its would-have-written count. glibc fortification checks selected calls when size information is available; it does not make every overflow safe.

C-STR-02 strncpy leaves no terminator

SR-404 · Dashboard: fault code (DTC)

✕ BUGGY
```c
char code[4];
strncpy(code, "P0301", sizeof code);  // copies "P030", no '\0'
printf("DTC %s\n", code);  // not a valid terminated string for %s
```

▶ gcc -Wstringop-truncation: 'strncpy' output truncated copying 4 bytes …

What goes wrong: strncpy copies up to n characters. If no null occurs among those n characters it supplies no terminator; a shorter source is padded to n bytes, not necessarily the whole destination allocation.

> FIX
```c
char code[6];  // five characters plus the terminating null
int n = snprintf(code, sizeof code, "%s", "P0301");
if (n < 0 || (size_t)n >= sizeof code) return 1;
printf("DTC %s\n", code);
```

Avoid it: Choose string formatting or fixed-width field copying deliberately. An n-1 plus explicit-null idiom requires positive capacity and valid storage; it may truncate. GCC14 documents -Wstringop-truncation. ASan reports selected executed invalid accesses, not every possible over-read.

Sources: C11 draft 7.24.2.4;7.21.6.5; GCC14 docs Wstringop-truncation; Clang docs Introduction;Limitations; Host example capture C-STR-02 exact supplied-example diagnostics/results

C-STR-03 Missing null terminator

SR-404 · SR-404 badge text

✕ BUGGY
```c
char badge[3] = "404";   // legal C: the '\0' is silently dropped
puts(badge);             // requires a terminated string
```

▶ ASan: stack-buffer-overflow

What goes wrong: C11 permits a character array to hold exactly the non-null characters of a string literal. That array is not a terminated C string; C++17 disallows the shown initializer. Raw binary input needs a terminator only when later used with C-string operations.

> FIX
```c
char badge[] = "404";    // compiler sizes it: 4 bytes incl. '\0'
puts(badge);
```

Avoid it: For this literal, infer the array length with []. For received text, check a successful nonnegative count and actual array capacity before adding a null. A pointer sizeof is not buffer capacity. GCC15 documents -Wunterminated-string-initialization in -Wextra.

Sources: C11 draft 6.7.9p14;7.1.1p1; C++17 draft N4659 [dcl.init.string]p2; GCC15 docs Wunterminated-string-initialization; Host example capture C-STR-03 exact supplied-example diagnostics/results

C-STR-04 Format string from data

SR-404 · Radio / CB messages

✕ BUGGY
```c
void show_cb(const char *msg) {
    printf(msg);  // data must not become conversion instructions
}
```

▶ gcc -Wformat-security: format not a string literal and no format argum…

What goes wrong: A format string controls conversions and required arguments. Passing untrusted text as the format can violate that contract; missing or incorrectly typed arguments are undefined. Neither a particular memory leak nor remote code execution is guaranteed by this example.

> FIX
```c
int show_cb(const char *msg) {
    return fputs(msg, stdout) == EOF ? -1 : 0;
}
```

Avoid it: Pass message text as data, as in fputs or printf("%s", msg), with a valid terminated string. Requiring literal formats is project guidance. GCC14 -Wformat=2 requests additional nonliteral/security checks; GCC format(printf,1,2) suits a wrapper with format first and checked arguments starting second.

Sources: C11 draft 7.21.6.1p8-p9;7.21.7.4; GCC14 docs Wformat=2;Wformat-security;Wformat-nonliteral; GCC14 docs format(archetype,string-index,first-to-check); Host example capture C-STR-04 exact supplied-example diagnostics/results; Defined host checks C-STR-04 defined inputs/error policy; host_abi observations

C-STR-05 Mismatched printf specifiers

SR-404 · Steering readout (Ep 2)

✕ BUGGY
```c
double angle = 12.5;
size_t n = 3;
printf("%d deg, %d samples\n", angle, n);  // UB: wrong types
```

▶ gcc -Wformat=: format '%d' expects argument of type 'int', but argumen…

What goes wrong: The format specifies required argument types after the applicable promotions. A double supplied for %d violates the contract. ISO C gives no portable garbage value, register/stack retrieval pattern or resulting steering motion.

> FIX
```c
double angle = 12.5;
size_t n = 3;
printf("%.1f deg, %zu samples\n", angle, n);
```

Avoid it: Match actual argument types: %zu for size_t, %lld for long long, and the corresponding inttypes macros where those typedefs exist. GCC14 -Wformat checks documented cases; treating warnings as errors is a build policy. This mismatch example is compiled for a diagnostic, not executed.

Sources: C11 draft 6.5.2.2;7.21.6.1p7-p9;7.8.1; GCC14 docs Wformat; Host example capture C-STR-05 exact supplied-example diagnostics/results

C-STR-06 gets()

SR-404 · Diagnostics console

✕ BUGGY

```
char line[32];
gets(line);                 // no length limit: removed in C11
```

▸ gcc error: implicit declaration of function 'gets'; did you mean 'fget…

What goes wrong: gets has no capacity parameter. C99TC3 deprecated it and C11 removed it from the standard library. GCC14 defaults make undeclared-function calls errors; library extensions and compiler options are separate from the language removal.

> FIX

```
char line[32];
if (fgets(line, sizeof line, stdin) == NULL)
    return ferror(stdin) ? 1 : 0;  // error or clean empty
EOF
char *nl = strchr(line, '\n');
if (nl != NULL) *nl = '\0';
else {
    int next = getchar();
    if (next != '\n' && next != EOF) return 1; // too long
    if (next == EOF && ferror(stdin)) return 1;
}
puts(line);
```

Sources: C11 draft Foreword: gets removed;7.21.7.2;7.21.7.6;7.21.10.3;7.24.5.2; C99TC3 draft N1256 7.19.7.7;7.26.9; GCC14 docs Implicit function declarations; Host example capture C-STR-06 exact supplied-example diagnostics/results; Defined host checks C-STR-06 defined inputs/error policy; host_abi observations

Avoid it: This one-command reader accepts NUL-free text that fits 31 characters, strips a newline and rejects a longer line. It checks EOF/error before using the buffer. A repeated-command reader also needs a resynchronization policy after rejection; selecting C17 alone does not prohibit every libc extension.

C-STR-07 scanf without width or result check

SR-404 · Throttle input (Ep 3)

✕ BUGGY

```
int main(void) {
    char name[16]; int pedal;
    scanf("%s %d", name, &pedal); // no width or result check
    printf("%s %d\n", name, pedal);
    return 0;
}
```

▸ ASan: stack-buffer-overflow

What goes wrong: An unbounded %s can exceed the destination for a long token. A failed %d conversion does not assign the initially uninitialized pedal, and the nonmatching character can remain unread. A numeric result outside the destination range is undefined even if you check the conversion count afterward.

> FIX

```
int parse_pedal(const char *line, char name[16], int *pedal)
{
    size_t n = strcspn(line, " \t\r\n\v\f");
    if (n == 0 || n >= 16 || line[n] == '\0') return -1;
    const char *p = line + n;
    char *end;
    errno = 0;
    long v = strtol(p, &end, 10);
    if (errno || end == p || v < INT_MIN || v > INT_MAX)
        return -1;
    while (isspace((unsigned char)*end)) ++end;
    if (*end != '\0') return -1;
    memcpy(name, line, n);
    name[n] = '\0';
    *pedal = (int)v;
    return 0;
}
```

Sources: C11 draft 7.21.6.2p10,p12,p16;7.21.7.2;7.22.1.4;7.4;7.24.2.1;7.24.5.3; Clang docs Alias declaration; Clang docs Selected functions and strtol-family exclusion; Host example capture C-STR-07 exact supplied-example diagnostics/results; Defined host checks C-STR-07 defined inputs/error policy; host_abi observations

Avoid it: The fixed parser takes a terminated text line, a name buffer of at least 16 bytes and a valid output pointer. Its policy is one leading word followed by one representable int, with trailing whitespace allowed. The driver accepts NUL-free text and rejects overlong lines. Current cert-err34-c aliases a selected-call check that explicitly excludes strtol-family calls; validate those calls yourself.

C-STR-08 Unchecked fopen / fprintf / fclose

SR-404 · Trip computer log

✕ BUGGY

```
void save_trip(const char *path, long odo) {
    FILE *f = fopen(path, "w");     // may be NULL
    fprintf(f, "odo=%ld\n", odo);  // NULL violates the FILE
argument contract
    fclose(f);                      // flush errors ignored
}
```

▸ UBSan: null pointer passed as argument 1, which is declared to never b…

What goes wrong: fopen can fail, and buffered writes can report failure at writing, flushing or closing. Ignoring failures can hide data loss; ignoring a successful return does not itself lose data. Successful library calls alone do not prove durable storage.

> FIX

```
int save_trip(const char *path, long odo) {
    FILE *f = fopen(path, "w");
    if (f == NULL) return -1;
    int ok = fprintf(f, "odo=%ld\n", odo) >= 0;
    if (fclose(f) != 0) ok = 0; // stream is closed even on
error
    return ok ? 0 : -1;
}
```

Avoid it: Check relevant open/write/close results and keep an error policy. A failed fclose still invalidates the stream. The Linux /dev/full fixture fails writes with ENOSPC; not every operation on it fails. GCC warn_unused_result and analyzer checks have compiler/path scope; C23 nodiscard recommends a diagnostic rather than guaranteeing every warning.

Sources: C11 draft 7.21.3;7.21.5.1-7.21.5.3;7.21.6.1p14;7.21.4.1; POSIX2024 fflush return/errors; Linux source v6.6 drivers/char/mem.c write_full; C23 draft N3096 6.7.12.2p4-p5 recommended diagnostics; GCC14 docs warn_unused_result; GCC14 docs Wanalyzer-null-argument; Host example capture C-STR-08 exact supplied-example diagnostics/results

C-STR-09 Ignoring short writes

SR-404 · Serial byte stream

✕ BUGGY

```
int send_frame(int fd, const char *buf, size_t n) {
    return write(fd, buf, n) < 0 ? -1 : 0;  // ignores short
writes
}
```

▸ prints: rc=0 sent=65536 of 1048576

What goes wrong: POSIX write reports bytes transferred and can return a positive count smaller than requested. For this byte-stream helper, that is partial progress, not completion. fwrite instead returns complete item counts; datagram send has a different message-boundary contract.

> FIX

```
int write_all(int fd, const char *buf, size_t n) {
    while (n > 0) {
        size_t chunk = n > (size_t)SSIZE_MAX ?
(size_t)SSIZE_MAX : n;
        ssize_t w = write(fd, buf, chunk);
        if (w < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (w == 0) return -1;
        buf += w;
        n -= (size_t)w;
    }
    return 0;
}
```

Sources: POSIX2024 Partial bytes;SSIZE_MAX;EINTR;EAGAIN; C11 draft 7.21.8.2p3 fwrite item count; POSIX2024 Message boundaries;EMSGSIZE; POSIX2024 Read count and EINTR; POSIX2024 Driver pipe ends; POSIX2024 Driver start argument/return; POSIX2024 Driver thread termination and join return; Host example capture C-STR-09 exact supplied-example diagnostics/results; Defined host checks C-STR-09 defined inputs/error policy; host_abi observations

Avoid it: The fixed helper is for valid buffers and blocking byte streams. It retries EINTR and rejects zero progress; other errors are returned. Nonblocking I/O needs a readiness/cancellation policy. Do not apply this loop blindly to datagrams or fwrite items. Test partial progress and error paths separately.

C-STR-10 atoi() with no error checking

SR-404 · Config: speed limit

✕ BUGGY

```c
int parse_limit(const char *arg) {
    return atoi(arg);  // no error report; an unrepresentable result is UB
}
```

▸ prints: limit=0

What goes wrong: atoi cannot distinguish conversion errors through its return. No-conversion text produces 0, but a valid numeric prefix before trailing junk can produce a nonzero value. A result unrepresentable as int is undefined; a particular speed-limit effect requires the application policy.

> FIX

```c
int parse_limit(const char *arg, int *out) {
    if (arg == NULL || out == NULL) return -1;
    char *end;
    errno = 0;
    long v = strtol(arg, &end, 10);
    if (errno || end == arg || *end != '\0' || v < 0 || v > 255)
        return -1;
    *out = (int)v;
    return 0;
}
```

Sources: C11 draft 7.22.1p1;7.22.1.2;7.22.1.4; Clang docs Alias declaration; Clang docs Selected calls;strtol-family excluded; Host example capture C-STR-10 exact supplied-example diagnostics/results; Defined host checks C-STR-10 defined inputs/error policy; host_abi observations

Avoid it: The example policy accepts values 0..255, rejects trailing characters and leaves the output unchanged on failure. Check errno, end pointer and range before narrowing. Current cert-err34-c is an alias for a selected conversion-call check; its exclusion of strtol does not certify your error handling.

C-STR-11 getchar() stored in a char

SR-404 · Serial console

✕ BUGGY

```c
char c;                          // narrowing loses the int result
int n = 0;
while ((c = getchar()) != EOF)  // captured 8-bit signed-char/EOF=-1 target
    n++;
printf("%d bytes\n", n);
```

▸ prints: 2 bytes

What goes wrong: getchar returns int, including the negative EOF constant. C11 does not require EOF=-1 or 8-bit bytes. On this captured 8-bit signed-char host, narrowing 0xFF loses the distinction used by the loop. Other representations/conversion choices need their own analysis.

> FIX

```c
int c;  // retain the library result for the EOF comparison
size_t n = 0;
while ((c = getchar()) != EOF) {
    if (n == SIZE_MAX) return 1;
    ++n;
}
if (ferror(stdin)) return 1;
printf("%zu bytes\n", n);
```

Avoid it: Test EOF before narrowing an input character and distinguish read errors when needed. Plain-char signedness is implementation-defined. Character classifiers require EOF or a value representable as unsigned char; after excluding EOF, convert an actual character before isdigit/isalpha. Positive out-of-domain values are invalid too.

Sources: C11 draft 5.2.4.2.1;6.2.5p15;6.3.1.3;7.21.1p3;7.21.7.1;7.21.7.6;7.21.10.3;7.4p1; Host example capture C-STR-11 exact supplied-example diagnostics/results; Defined host checks C-STR-11 defined inputs/error policy; host_abi observations

C-STR-12 Comparing strings with ==

SR-404 · Brake command parser

✕ BUGGY

```c
bool is_brake_cmd(const char *cmd) {
    return cmd == "BRAKE";     // compares addresses, not text
}
```

▸ gcc -Waddress: comparison with string literal results in unspecified b…

What goes wrong: Pointer equality follows pointer-value rules, not string-content rules. Literal storage may be shared, so a coincidental equal pointer does not validate a text comparison algorithm.

> FIX

```c
bool is_brake_cmd(const char *cmd) {
    return cmd != NULL && strcmp(cmd, "BRAKE") == 0;
}
```

Avoid it: For valid terminated strings use a content comparison; use explicit lengths for other data. strncmp can establish a matching prefix without proving whole-string equality. GCC14 documents -Waddress under -Wall for selected literal-address comparisons; a warning is not complete validation.

Sources: C11 draft 6.4.5p7;6.5.9p6;7.24.4.1;7.24.4.2;7.24.4.4; GCC14 docs Waddress;Wall; Host example capture C-STR-12 exact supplied-example diagnostics/results; Defined host checks C-STR-12 defined inputs/error policy; host_abi observations

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/`. Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.